

IQA Phase 2 Recipe Report

Validation recipe, evidence and operational commands

Document type	IQA Phase 2 Recipe Report
Version	1.0
Status	Final Phase 2 update, JAL02, JAL03 and JAL04 validated. IQA2_LIV03 remains open only for notebook update.
Date	17 June 2026
Scope	IQA MLOps Phase 2, validated JAL02, JAL03 and JAL04, remaining IQA2_LIV03 notebook update
Confidentiality	Internal project documentation

Final update after server side PostgreSQL validation of IQA2_JAL02 and project confirmation that IQA2_JAL03 and IQA2_JAL04 are validated.

Contents

Section	Title
1	Executive summary
2	Validation environments
3	GitHub and repository architecture
4	Validation recipe
5	JAL02 final validation recipe
6	JAL03 and JAL04 closure status
7	IQA2_LIV03 remaining notebook update
8	Operational handover
9	Evidence register

1. Executive summary

This final recipe report replaces the earlier v0.9 draft. IQA2_JAL02 is now validated. It was validated on the shared IQA server with live PostgreSQL metadata persistence. The project team also confirmed that IQA2_JAL03 and IQA2_JAL04 are validated. The remaining work to close IQA2_LIV03 is the notebook update and final alignment of written deliverables.

Final Phase 2 status

JAL02 validated at 100%, JAL03 validated, JAL04 validated. IQA2_LIV03 remains open only because the notebooks still need to be updated.

Validation layer	Result	Evidence
CubeAI full baseline	464 passed, 11 skipped, 1 warning	464 passed, 11 skipped, 1 warning after the Phase 2 origin/main resync on CubeAI.
IQA server PostgreSQL live	9 passed	POSTGRES_CONNECTION_OK iqa_metadata iqa; IQA metadata schema is initialized; 9 passed in 6.14s.
Project milestone status	JAL03 and JAL04 validated	Confirmed by project planning update and team status.
Remaining deliverable action	Notebook update	IQA2_LIV03 closure requires notebook refresh.

2. Validation environments

Environment	Path or scope	Purpose
CubeAI	/mnt/nvme/mlops/iqa-mlops	Development, branch integration, local baseline and broad pytest validation.
IQA server	/opt/iqa/iqa-mlops and detached /tmp worktrees	Shared runtime validation against origin/main without modifying the live repository tree.
GitHub	data-fablab/iqa-mlops	Pull request merge history, repository architecture and final main branch reference.
Server runtime	Docker Compose stack	API, inference, Streamlit, Airflow, PostgreSQL, MLflow, MinIO, Prometheus, Grafana and reverse proxy.

The server validation uses a detached worktree from origin/main. This keeps the runtime check isolated from the currently checked out server branch and prevents accidental local modifications.

3. GitHub and repository architecture

The current repository layout was reviewed from GitHub main and from the server checkout. The repo is structured as a multi service MLOps project rather than a single notebook repository.

Area	Repository items	Purpose
Workflows and orchestration	.github/workflows, airflow/dags	CI/CD entry point and Airflow DAG orchestration.
Application and services	src/iqa, Dockerfile, deploy	FastAPI, inference service, metadata repository, runtime services and compose deployment.
Data and model lifecycle	data, configs, dvc.yaml, models/manifests	Replay plans, manifests, candidate datasets, DVC and model metadata.
Operational docs and evidence	docs, reports, scripts, tests	Runbooks, governance documents, validation commands and pytest contracts.

Scraped repository folders and files: .dvc, .github/workflows, airflow/dags, configs, data, deploy, docs, models/manifests, reports, scripts, src/iqa, tests, Dockerfile, dvc.yaml, pyproject.toml and README.md.

4. Validation recipe

The broad Phase 2 baseline validates API contracts, AI security controls, metadata repository contracts, replay and lifecycle tests. The latest broad baseline after repository resynchronization produced 464 passed, 11 skipped and 1 warning on CubeAI.

Standard command family

```
python -m pytest tests -q
python -m pytest tests/api tests/security tests/contracts tests/metadata -q
python -m pytest tests/api/test_replay_runs_contract.py tests/api/test_postgres_write_through_contract.py
tests/metadata/test_postgres_metadata_foundation.py -q -rs
```

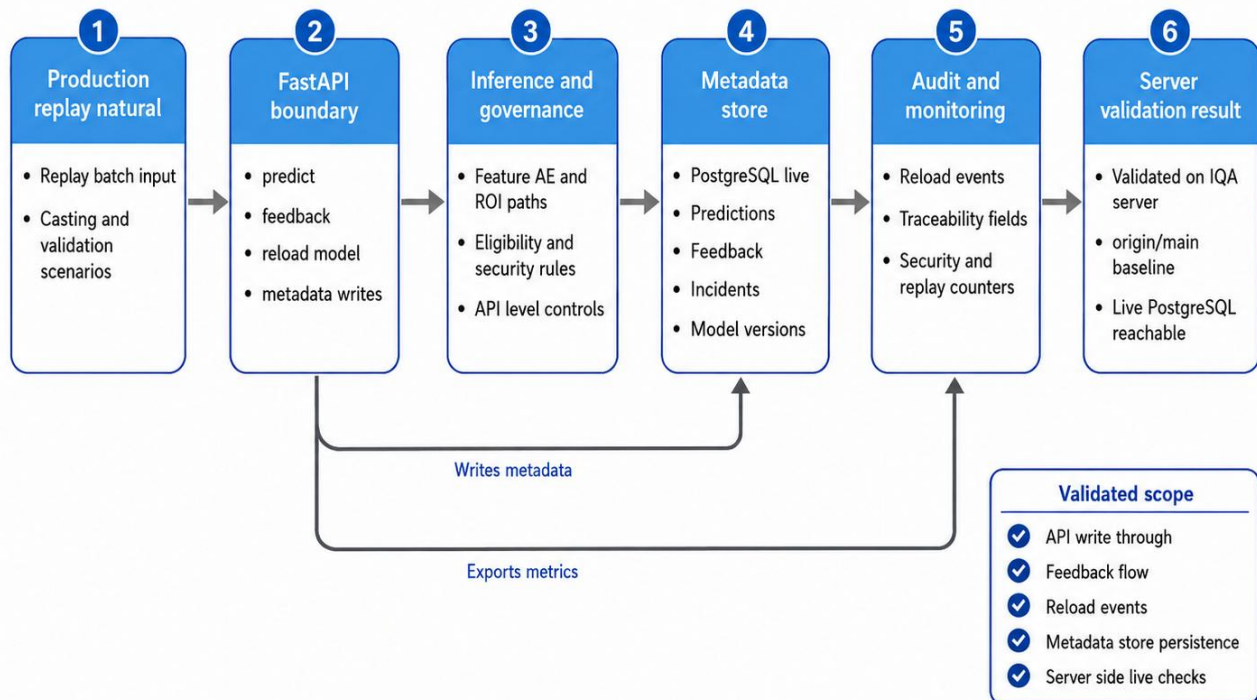
5. JAL02 final validation recipe

JAL02 proves that `production_replay_natural` passes through the API, feedback, reload events and the PostgreSQL metadata store. The final validation was run on the IQA server with the PostgreSQL URL adapted from Docker network host `postgres` to host side `127.0.0.1`.

Step	Command or check	Result
Load server environment	<code>source /opt/iqa/iqa-mlops/.env</code> and adapt <code>IQA_METADATA_DB_URL</code> host	Variables loaded without printing secrets.
Connect PostgreSQL	<code>psycopg.connect(IQA_METADATA_DB_URL)</code>	<code>POSTGRES_CONNECTION_OK</code> <code>iqa_metadata iqa.</code>
Initialize schema	<code>scripts/init_metadata_db.py</code>	IQA metadata schema is initialized.
Run live contracts	<code>test_postgres_write_through_contract.py</code> and <code>test_postgres_metadata_foundation.py</code>	9 passed in 6.14s.

IQA2 JAL02 validation schema

Replay naturel branché sur API



Outcome: IQA2_JAL02 validated at 100% on the server side baseline.

6. JAL03 and JAL04 closure status

Project status is now updated: IQA2_JAL03 and IQA2_JAL04 have passed and are validated. The recipe therefore treats these milestones as closed for Phase 2 reporting. Any remaining action belongs to deliverable packaging rather than core milestone validation.

Milestone	Status	Reporting consequence
IQA2_JAL02	Validated at 100%	Natural replay through API, feedback, reload events and PostgreSQL live can be reported as closed.
IQA2_JAL03	Validated	Model lifecycle validation can be described as completed for Phase 2.
IQA2_JAL04	Validated	Phase 2 platform delivery gate can be described as completed.
IQA2_LIV03	Open	Notebook update still required before final closure.

7. IQA2_LIV03 remaining notebook update

The notebooks are the remaining artifact to align with the final Phase 2 reality. They must describe JAL02 as validated and must reflect that JAL03 and JAL04 are validated.

- Replace all old JAL02 open status wording with server side PostgreSQL live validation evidence.
- Add JAL03 and JAL04 as validated milestones.
- Include the final Phase 2 architecture view and the GitHub repository architecture view.
- Move Kubernetes to a Phase 4 scalability target, not Phase 3.
- Add Kong API Gateway as a Phase 3 security and routing workstream.

8. Operational handover

Rule	Reason
Do not validate replay against memory only state	JAL02 was explicitly validated against live PostgreSQL metadata persistence.
Keep FastAPI as governance boundary	Inference should not write directly to metadata. FastAPI enforces schemas, audit and incidents.
Keep MLflow Registry as source of truth	MinIO is an artifact store, not the promotion authority.
Use server worktrees for final validation	Detached worktrees avoid modifying the live server checkout.

9. Evidence register

Evidence	Key result
iqa_jal02_server_live_postgres_host_20260617_143652.txt	POSTGRES_CONNECTION_OK iqa_metadata iqa; schema initialized; 9 passed.
iqa_jal02_full_baseline_20260617_141214.txt	464 passed, 11 skipped, 1 warning on CubeAI.
iqa_phase3_server_specs_20260617_172308.txt	16 CPU, 39 GiB RAM, RTX 3060, Docker Compose services running and low load.
GitHub data-fablab/iqa-mlops main	MLOps repo with workflows, Airflow, deploy, docs, src, tests, DVC and Docker artifacts.